

MetaXtract: Technical Design, Data Extraction, and Outputs

1 Overview

MetaXtract extracts technical metadata, scan-level information, peak lists, runtime metrics, and SDRF-Proteomics metadata from Thermo Fisher RAW mass spectrometry files. The software can be used through a graphical user interface (GUI), a command-line interface (CLI), containerized Docker or Apptainer/Singularity environments, and a portable Snakemake workflow for local or HPC batch execution.

The GUI is intended for interactive use, including manual curation of SDRF-Proteomics sample metadata. The CLI and workflow modes are intended for reproducible batch processing and can be configured either with command-line arguments or a YAML configuration file. Containerized execution provides a portable Linux runtime for Docker, WSL2, and HPC infrastructures. Extraction of vendor method text is available where supported by the Thermo RAW access layer; on Linux/container runtimes, method-text extraction can be disabled while the remaining metadata, scan-header, peak-list, runtime, and SDRF draft workflows remain available.

2 Installation and Running

2.1 Requirements

For native Windows and Linux execution, MetaXtract requires Python 3.9 or newer and Thermo Fisher RAW access through the RawFileReader DLLs included in this repository. The same Python entry point is used by the GUI and CLI.

On macOS, use the containerized CLI workflow. macOS cannot run Linux containers directly, so Docker Desktop or another VM-backed Docker engine is required. The GUI is not part of the container workflow; use command-line options or a YAML configuration file in that mode.

2.2 Native Windows and Linux Installation

Install the Python dependencies from the repository requirements file:

```
pip install -r requirements-cli.txt
```

The equivalent explicit dependency list is:

```
pip install numpy pandas pyarrow pyyaml tqdm plotly anndata h5py \
pythonnet PySide6
```

Run the GUI on Windows or Linux with:

```
python main.py
```

Run the CLI on Windows or Linux with a YAML configuration file:

```
python main.py --config /path/to/config.yml
```

2.3 Building a Windows Executable

Install PyInstaller:

```
pip install pyinstaller
```

If PyQt6 is installed in the same environment, exclude it from the build so that PyInstaller collects the intended PySide6 GUI stack:

```
pyinstaller --noconfirm --onedir --console --name MetaXtract \  
--icon assets/icon.ico \  
--add-data "os_data;os_data" \  
--add-data "assets;assets" \  
--hidden-import=clr \  
--hidden-import=System \  
--hidden-import=pythonnet \  
--hidden-import=matplotlib.backends.backend_pdf \  
--hidden-import=matplotlib.backends \  
--collect-all PySide6 \  
--collect-all h5py \  
--collect-submodules h5py \  
--exclude-module PyQt6 \  
--exclude-module PyQt6.sip \  
--exclude-module PyQt6.QtCore \  
--exclude-module PyQt6.QtGui \  
--exclude-module PyQt6.QtWidgets \  
main.py
```

2.4 Docker and macOS

MetaXtract can be run as a CLI-only Linux container. This is the recommended mode for macOS and useful anywhere a reproducible Linux runtime is preferred. On macOS, install and start Docker Desktop before building the image:

```
brew install --cask docker  
open -a Docker  
docker info
```

Build the Docker image from the MetaXtract repository:

```
docker build -t metaxtract:latest .
```

Run a mounted RAW-file directory and output directory:

```
docker run --rm \  
-v /path/to/raw_files:/data:ro \  
-v /path/to/output:/out \  
metaxtract:latest \  
--input /data/sample.raw \  
--output-dir /out
```

```

--file-based-details \
--complete-ms1 \
--complete-ms2 \
--ms1-peaklist-export \
--ms2-peaklist-export \
--graphical-representation

```

Run the bundled small RAW file:

```

mkdir -p output/docker_small
docker run --rm \
  -v "$(pwd)/data:/data:ro" \
  -v "$(pwd)/output/docker_small:/out" \
  metaxtract:latest \
  --input /data/small.RAW \
  --output-dir /out \
  --file-based-details \
  --complete-ms1 \
  --complete-ms2 \
  --ms1-peaklist-export \
  --ms2-peaklist-export \
  --graphical-representation

```

Run Docker with the included container-ready configuration:

```

mkdir -p output/docker_small
docker run --rm \
  -v "$(pwd)/data:/data:ro" \
  -v "$(pwd)/output/docker_small:/out" \
  -v "$(pwd)/config_container_small.yml:/config.yml:ro" \
  metaxtract:latest \
  --config /config.yml

```

On Apple Silicon Macs, build and run without `--platform` first so Docker uses the native ARM64 Linux backend:

```

docker build -t metaxtract:latest .
mkdir -p output/docker_small
docker run --rm \
  -v "$(pwd)/data:/data:ro" \
  -v "$(pwd)/output/docker_small:/out" \
  metaxtract:latest \
  --input /data/small.RAW \
  --output-dir /out \
  --file-based-details \
  --complete-ms1 \
  --complete-ms2 \
  --ms1-peaklist-export \
  --ms2-peaklist-export \
  --graphical-representation

```

Running the container with `--platform linux/amd64` on Apple Silicon uses emulation. If Mono or pythonnet crashes with a message such as `Assertion: should not be reached at tramp-amd64.c`, rebuild and run without `--platform`. If native ARM64 execution does not work on the machine, run the container on an x86_64 Linux workstation or HPC node instead.

2.5 Apptainer/Singularity on WSL2 and HPC

For HPC systems, build an Apptainer/Singularity image from Docker or from the included definition file. On WSL2 Ubuntu, Apptainer can be installed with:

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:apptainer/ppa
sudo apt update
sudo apt install -y apptainer
```

Check the installation:

```
apptainer --version
```

If the PPA is unavailable for the installed Ubuntu version, check the Ubuntu release with:

```
lsb_release -a
```

Build the Apptainer/Singularity image and run a mounted RAW-file directory:

```
apptainer build metaxtract.sif apptainer.def
apptainer run \
  --bind /path/to/raw_files:/data,/path/to/output:/out \
  metaxtract.sif \
  --input /data/sample.raw \
  --output-dir /out \
  --file-based-details
```

Run the bundled small RAW file with Apptainer/Singularity on WSL2 or HPC:

```
mkdir -p output/hpc_small
apptainer run \
  --bind "$(pwd)/data:/data","$(pwd)/output/hpc_small:/out" \
  metaxtract.sif \
  --input /data/small.RAW \
  --output-dir /out \
  --file-based-details \
  --complete-ms1 \
  --complete-ms2 \
  --ms1-peaklist-export \
  --ms2-peaklist-export \
  --graphical-representation
```

Run Apptainer/Singularity with the included configuration file:

```
mkdir -p output/hpc_small
apptainer run \
  --bind "$(pwd)/data:/data", "$(pwd)/output/hpc_small:/out" \
  --bind "$(pwd)/config_container_small.yml:/config.yml" \
  metaxtract.sif \
  --config /config.yml
```

The container uses the bundled Thermo RawFileReader DLLs through Mono and **pythonnet**. MS-method and LC-method export should be disabled on Linux containers because those Thermo method-reading calls are not supported on Linux.

2.6 Workflow Execution

The Snakemake workflow provides batch execution for multiple files and PRIDE-style datasets. It reads a YAML configuration file, prepares inputs, and delegates extraction to the same MetaXtract CLI. Depending on the workflow configuration and execution environment, it can run locally, through Docker, or through Apptainer/Singularity.

3 Execution Modes

Table 1: Current MetaXtract execution modes.

Mode	Use case	Notes
GUI	Interactive extraction and metadata curation.	Supports file selection, output selection, scan-header column selection, visualisation, runtime reporting, and SDRF-Proteomics metadata editing.
CLI	Scripted or batch extraction from one or more RAW files.	Accepts direct command-line options or a YAML configuration file. Supports file-based details, scan headers, peak lists, visualisations, runtime TSV logging, SDRF draft generation, and final SDRF generation from a curated TSV.
Docker	Portable CLI execution on systems with Docker Desktop or another Docker engine.	Useful for Linux and macOS users who want a reproducible Linux runtime. On macOS, Docker Desktop or another VM-based backend is required because macOS cannot run Linux containers directly.
Apptainer/ Singularity	Container execution on WSL2 and HPC systems.	Useful on clusters where Docker is unavailable or not permitted. The Apptainer/Singularity definition uses the same CLI entry point as Docker.
Snakemake workflow	Portable workflow execution for multiple files and PRIDE-style datasets.	The workflow reads a YAML configuration file, prepares inputs, runs MetaXtract locally or through Docker/Apptainer/Singularity, and summarizes runtime outputs.

4 Technical Architecture and Data Flow

4.1 RAW access layer

The extraction core is the Python `MetaXtract` class in `raw_parser.py`. It uses `pythonnet` to load Thermo Fisher’s `CommonCore.Data` and `CommonCore.RawFileReader` .NET assemblies. At start-up, the bridge forces an invariant US-English numeric culture so that decimal values returned through .NET are parsed consistently. For each input, `RawFileReaderAdapter.FileFactory` opens the native RAW file, MS controller 1 is selected, and run-header values such as the first and last scan, mass range, acquisition-time range, resolution, and spectrum count are cached. Numeric .NET arrays are pinned, copied into NumPy arrays, and then released; the copy prevents later .NET garbage collection from invalidating Python-side data.

The parser deliberately separates vendor access from orchestration. The CLI and PySide6 GUI use the same parser, output conventions, and shared Parquet, H5AD, Plotly, runtime, and SDRF components. The CLI processes files sequentially. The GUI performs extraction in a worker thread so that progress, log messages, and cancellation remain responsive. Docker and Apptainer add Python 3.11 and Mono around this same entry point; the Snakemake workflow creates one per-sample configuration and invokes that CLI locally or in a selected container runtime.

4.2 Per-file processing pipeline

For each RAW file, processing follows these stages:

1. Open the file, select its MS controller, and create a sample-specific output directory.
2. Traverse scan numbers from 1 through `SpectraCount`. The scan event supplies the MS order, while scan statistics identify centroid and profile data. Centroid MS2, profile MS2, and MS1 counts and scan-number lists are maintained separately.
3. Resolve requested scalar fields. `MetaXtract` first inspects the scan’s trailer-extra dictionary using the current output name and known legacy aliases. If no trailer value exists, a typed `RawFileReader` accessor supplies the value; unresolved values are written as N/A.
4. Serialize the selected products. Scan-header CSV rows are streamed as scans are visited. Peak arrays are written to Parquet in blocks of 100 spectra using Snappy compression. `AnnData` export stores scalar scan metadata in `obs` and variable-length spectral arrays in HDF5.
5. Close the RAW handle, record success/failure status, elapsed time, and resident memory, then continue with the next file.

Cancellation is cooperative: GUI stop requests and CLI termination signals set an event that is checked between scans and output stages. This avoids abandoning the .NET RAW handle in the middle of normal control flow.

4.3 Scan selection and fallback rules

The scan populations are not identical for every output. MS1 scan headers and MS1 Parquet output accept both profile and centroid MS1 scans. A profile MS1 spectrum is read from the segmented scan, whereas a centroid MS1 spectrum uses the centroid stream. MS2 scan headers, the extended MS2 Parquet file, and the H5AD export are restricted to centroid MS2 scans. Profile MS2 scans are counted in the file summary but are not written to these centroid-oriented products.

Selected-ion intensity receives additional handling because it is not populated uniformly across acquisition methods. A finite trailer value named **Selected Ion Intensity** (or the legacy alias **Precursor Intensity**) has priority. Otherwise, MetaXtract locates the precursor/master scan, obtains the monoisotopic precursor m/z, and uses the corresponding point in the centroid or segmented precursor spectrum. Reports retain the provenance as **trailer**, **computed**, or **missing**.

4.4 Serialization schemas and resource behaviour

Table 2: Technical representation of scan-level outputs.

Product	Row/unit	Stored representation
Scan-header CSV	One selected scan	Scan number and RAW basename plus user-selected scalar columns. Values are resolved from trailer metadata first and typed accessors second.
Technical-details CSV	One scan of the requested MS order	The union of cleaned trailer-extra keys across scans. Control characters, surrounding whitespace, and trailing colons are removed; divider labels are dropped; numeric strings are coerced; duplicate keys receive #2, #3, and subsequent suffixes. scan_number , is_centroid , mz_array , and intensity_array . Arrays are variable-length lists.
MS1 Parquet	One MS1 spectrum	scan_number plus variable-length m/z, intensity, resolution, noise, baseline, and charge arrays. If only a segmented spectrum is available, the four auxiliary arrays are filled with NaN values of matching length.
MS2 Parquet	One centroid MS2 spectrum	Scalar fields include scan number, retention time, TIC, peak count, precursor intensity, charge, injection time, base peak, and point count. The AnnData matrix has zero variables; m/z and intensity spectra are stored as variable-length float64 datasets under /uns/spectra/ .
MS2 H5AD	One centroid MS2 spectrum per obs row	

4.5 Technical-details CSV extraction

The technical-details export is a scan-level dump of Thermo trailer-extra metadata. It is intentionally less interpreted than the scan-header export: MetaXtract does not assign meanings to these fields beyond preserving the labels returned by the RAW access layer after light cleaning.

When **ms1_technical_details_export** or **ms2_technical_details_export** is enabled, the CLI and GUI use the same procedure:

1. Iterate from scan 1 through **NumSpectra**.
2. Read the scan MS order with **GetMSOrder(scanNumber)** and keep only scans matching the requested level, either MS1 or MS2.
3. Call **GetMoreMSInfos(scanNumber)** for each retained scan.
4. Inside **GetMoreMSInfos**, call **GetTrailerExtraInformation(scanNumber)** from **Thermo RawFileReader** and combine the returned **Labels** and **Values**.

5. Clean each label by removing control characters, collapsing repeated whitespace, trimming surrounding whitespace, and removing one trailing colon. Divider labels made only from surrounding = characters are skipped.
6. Convert integer-looking and floating-point-looking value strings to Python numbers where possible. Empty values become empty cells in the CSV.
7. Preserve duplicate cleaned labels by suffixing later occurrences as #2, #3, and so on.
8. Build the output columns dynamically from the union of labels observed across the retained scans. Missing values for a scan are written as N/A.

The resulting files are named `<sample>_technical_details_ms1_<timestamp>.csv` and `<sample>_technical_details_ms2_<timestamp>.csv`. Each row contains Scan Number, RAW File, and the trailer-extra columns observed for that MS order. In the complete example run on `MS2_MS1_orbitrap.raw`, the MS1 technical-details CSV contained 77 scan rows and the MS2 technical-details CSV contained 68 scan rows. Both files exposed the following trailer-extra labels:

Table 3: Trailer-extra labels observed in the complete MS2_MS1_orbitrap.raw technical-details export.

Column group	CSV columns
Row identifiers	Scan Number, RAW File.
Scan description and acquisition control	Scan Description, AGC, Micro Scan Count, Ion Injection Time (ms), Elapsed Scan Time (sec), Average Scan by Inst, Orbitrap Resolution, API Process Delay, Dependency Type, Multi Inject Info, Master Scan Number.
Precursor and isolation fields returned by the trailer	Monoisotopic M/Z, Charge State, HCD Energy, HCD Energy eV, MS2 Isolation Width, SPS Masses, SPS Masses Continued.
Conversion and correction fields returned by the trailer	Access ID, Conversion Parameter I, Conversion Parameter A, Conversion Parameter B, Conversion Parameter C, Conversion Parameter D, Conversion Parameter E, Temperature Comp. (ppm), RF Comp. (ppm), Space Charge Comp. (ppm), Resolution Comp. (ppm).
Lock-mass, raw-overfill, injection, and FAIMS labels returned by the trailer	Number of LM Found, LM Correction (ppm), Raw0vFtT, Injection t0, Reagent Ion Injection Time (ms), FAIMS Voltage On, FAIMS CV.

For S scans and P stored spectral points, scan classification and scalar metadata passes are $O(S)$, while peak extraction and serialization are $O(P)$. Selecting several products can cause several passes over the RAW file. Parquet buffers at most 100 spectrum records before writing a row group. By contrast, the current H5AD exporter retains all selected MS2 arrays until the file is written, and the technical-details exporter retains its rows while constructing the union of dynamic columns. These two products can therefore require memory proportional to their selected dataset size. Runtime monitoring samples process RSS every 0.2 seconds and includes Python, native, and .NET allocations.

5 YAML Configuration

MetaXtract can be configured with a YAML file to avoid long command lines and to make runs reproducible. A typical configuration contains input/output paths, output selections, scan-header columns, visualisation settings, optional multi-sample comparison settings, and optional SDRF settings.

```
io:
  input:
    - data/small.RAW
  output_dir: output/cli_small

outputs:
  file_based_details: true
  ms_method: false
  lc_method: false
  ms2_peaklist_export: true
  ms1_peaklist_export: true
  ms2_technical_details_export: false
  ms1_technical_details_export: false
  hdf5_export: false

scan_header:
  MS1:
    select_all: true
    columns:
      Total Ion Current: true
      Scan Start Time (min): true
      Base Peak Intensity: true
  MS2:
    select_all: true
    columns:
      Total Ion Current: true
      Scan Start Time (min): true
      Selected Ion Intensity: true

visualisation:
  enabled: true
  format: html

sdrf:
  draft: true
  draft_output: metadata.sdrf.draft.tsv
  metadata: null
  output: metadata.sdrf.tsv

multi_comparison:
  enabled: false
```

6 Main Outputs

Table 4 summarizes the principal files written by MetaXtract. Outputs are grouped into file-level metadata, method files, scan-level tables, peak lists, visualisations, runtime metrics, and SDRF-Proteomics metadata.

Table 4: Summary of MetaXtract outputs and their contents.
<sample> denotes the RAW base name.

Output	Typical filename	Content and scope
File-based details	<sample>_info.tsv	Per RAW file. Key-value TSV containing RAW file metadata, user ID, acquisition date, instrument name/model/serial/software/hardware details, scan counts, run time range, mass range, resolution, intensity summaries, and available sample information.
MS method	<sample>_MS_method.txt	Per RAW file. Mass spectrometer acquisition method text exported from the RAW when supported by the platform/runtime.
LC method	<sample>_LC_method.txt	Per RAW file. Liquid chromatography method text exported from the RAW when supported by the platform/runtime.
MS1 scan-header table	<sample>_scan_header_ms1_*.csv	Per RAW file. Selected MS1 scan-level metadata columns, one row per MS1 scan.
MS2 scan-header table	<sample>_scan_header_ms2_*.csv	Per RAW file. Selected MS2 scan-level metadata columns, one row per centroided MS2 scan.
MS1 peak list	<sample>_ms1_peaklist.parquet	Per RAW file. Per-scan MS1 peak arrays, including m/z and intensity arrays with centroid/profile status.
MS2 extended peak list	<sample>_ms2_peaklist.parquet	Per RAW file. Per-scan MS2 peak arrays, including m/z, intensity, resolution, noise, baseline, and charge arrays.
MS2 AnnData export	<sample>_MS2.h5ad	Per RAW file. Optional MS2 export in AnnData HDF5 format for programmatic downstream analysis.
Technical detail tables	<sample>_technical_details_ms1_*.csv, <sample>_technical_details_ms2_*.csv	Per RAW file. Additional scan-level trailer metadata returned by <code>GetMoreMSInfos</code> .
Interactive visualisations	<sample>_MS1_report.html, <sample>_MS2_report.html, MS1_compare.html, MS2_compare.html	Per RAW file, with optional multi-file comparisons. Interactive Plotly reports for TIC, base peak intensity, total number of peaks, precursor intensity, and cross-sample overlay/boxplot comparisons.
Runtime log	runtime_summary_YYYYMMDD_HHMMSS.tsv	Per run. One row per processed RAW file containing status, runtime in seconds, start/end memory in GB, sampled peak memory in GB, and memory change in GB.

Continued on next page

Output	Typical filename	Content and scope
SDRF draft	<code>metadata.sdrf.draft.tsv</code>	Per run. Dataset-level SDRF-Proteomics draft with required columns present and only RAW-derived fields filled. User-only biological and experimental design fields remain blank for manual completion.
Final SDRF	<code>metadata.sdrf.tsv</code>	Per run. Dataset-level SDRF-Proteomics file generated from GUI-entered metadata or a completed CLI metadata TSV, enriched with RAW filename, acquisition date, instrument, SDRF annotation tool, SDRF version, and SDRF template.

7 Scan Header Column Names

MetaXtract uses PSI-MS/mzML-style names where a clear controlled vocabulary term exists. Thermo trailer-extra fields without a direct one-to-one PSI-MS scan-header term are retained with a `thermo_` prefix. Older Thermo labels are still recognized internally as source aliases, but current CSV exports use the names listed below.

7.1 MS2 Scan Header

Table 5: Current MS2 scan-header parameters.

Parameter	Meaning
Scan Number	Sequential scan index within the RAW file.
RAW File	RAW base name.
Total Ion Current	Total scan signal intensity.
Total Number of Peaks	Number of detected peaks in the scan.
Scan Start Time (min)	Chromatographic retention time in minutes.
Base Peak m/z	m/z of the most intense peak in the scan.
Base Peak Intensity	Intensity of the most intense peak in the scan.
Selected Ion Intensity	Precursor selected-ion intensity for the MS2 event. MetaXtract first uses the RAW trailer value when present; if unavailable, it computes a fallback value from the precursor/master scan and the selected precursor m/z. Plotly reports note when this fallback or missing-value path is used.
Experimental Precursor Monoisotopic m/z	Monoisotopic precursor m/z estimate selected for fragmentation.
Charge State	Assigned precursor charge state, when available.
Isolation Window Width (m/z)	Precursor isolation window width in m/z.
Scan Window m/z Range	Acquisition scan-window range or ranges.
Dissociation Method	Fragmentation or activation method, e.g. HCD, CID, or ETD.
Collision Energy	Collision energy setting reported by the instrument.
Normalized Collision Energy (%)	Normalized collision energy when Thermo reports NCE.
Collision Energy (eV)	Collision energy expressed in electronvolts when available.
Mass Analyzer Type	Mass analyzer used for the scan.
Detector Type	Detector used for the scan.
Ion Injection Time (ms)	Ion injection or accumulation time in milliseconds.
Sampling Frequency	Instrument-reported sampling frequency parameter.
Filter String	Thermo scan filter string.
Scan Mode	Scan mode reported by the RAW access layer.

Parameter	Meaning
FAIMS Compensation Voltage	FAIMS compensation voltage when available.
thermo.Number of Channels	Thermo-specific channel count.
thermo.AGC	Thermo-specific AGC target/setting.
thermo.Micro Scan Count	Number of microscans.
thermo.Elapsed Scan Time (sec)	Instrument-reported elapsed scan time.
thermo.Average Scan by Inst	Thermo-specific average-scan diagnostic.
thermo.Orbitrap Resolution	Orbitrap resolution setting.
thermo.API Process Delay	Thermo-specific processing delay.
thermo.Dependency Type	Thermo-specific scan dependency classification.
thermo.Multi Inject Info	Thermo-specific multi-injection metadata.
thermo.Master Scan Number	Master/precursor scan reference number.
thermo.Access ID	Thermo-specific access identifier.
thermo.Conversion Parameter I	Thermo-specific conversion parameter.
thermo.Conversion Parameter A--E	Thermo-specific conversion coefficients.
thermo.Temperature Comp. (ppm)	Temperature compensation term in ppm.
thermo.RF Comp. (ppm)	RF compensation term in ppm.
thermo.Space Charge Comp. (ppm)	Space-charge compensation term in ppm.
thermo.Resolution Comp. (ppm)	Resolution compensation term in ppm.
thermo.Number of LM Found	Number of lock-mass features detected.
thermo.LM Correction (ppm)	Lock-mass correction in ppm.
thermo.RawOvFtT	Thermo-specific diagnostic value.
thermo.Injection t0	Thermo-specific injection timing offset.
thermo.Reagent Ion Injection Time (ms)	Reagent-ion injection time when relevant.
thermo.FAIMS Voltage On	Flag indicating whether FAIMS voltage was enabled.

7.2 MS1 Scan Header

Table 6: Current MS1 scan-header parameters.

Parameter	Meaning
Scan Number	Sequential scan index within the RAW file.
RAW File	RAW base name.
Ion Injection Time (ms)	Ion injection or accumulation time in milliseconds.
Total Number of Peaks	Number of detected peaks in the MS1 scan.
Total Ion Current	Total MS1 scan signal intensity.
Scan Start Time (min)	Chromatographic retention time in minutes.
Base Peak Intensity	Intensity of the most intense peak in the scan.
Base Peak m/z	m/z of the most intense peak in the scan.
Scan Mode	Scan mode reported by the RAW access layer.
thermo.Multi Inject Info	Thermo-specific multi-injection metadata.
thermo.Multiple Injection	Flag indicating multiple-injection acquisition.

8 SDRF-Proteomics Support

MetaXtract supports SDRF-Proteomics metadata export using the `ms-proteomics v1.1.0` template. In the GUI, the user fills required biological and experimental fields in an editor before processing. This is necessary because RAW files do not contain reliable values for fields such as organism, organism part, biological replicate, label, digestion enzyme, fraction identifier, technical replicate, disease, treatment, or other study factors.

The SDRF editor starts with the required MS-proteomics columns and allows users to add optional or specialized columns from the local snapshot of the official SDRF column registry. Custom

`factor value[...] columns` can also be added for experimental design variables.

For CLI, Docker, Apptainer, and HPC runs, MetaXtract can write a draft SDRF file during the normal extraction:

```
python main.py \
  --input data/small.RAW \
  --output-dir output/cli_small \
  --file-based-details \
  --sdrf-draft
```

This writes `metadata.sdrf.draft.tsv`. The draft contains all required SDRF columns but fills only values that MetaXtract can safely derive: source and assay names from the RAW basename, `technology type`, data filename, acquisition date, recognized instrument model, SDRF annotation tool, SDRF version, and SDRF template. User-only biological and experimental design fields are left blank for manual completion.

CLI users can pass a completed metadata TSV with the `--sdrf-metadata` option. MetaXtract validates the required fields, enriches the rows with RAW-derived filename, acquisition date, and instrument information, and writes the final `metadata.sdrf.tsv`. Common shorthand values are normalized to controlled vocabulary strings: DDA, DIA, PRM, and SRM are mapped to PRIDE acquisition-method terms, and common cleavage agents such as Trypsin, Lys-C, and Chymotrypsin are mapped to PSI-MS terms. Common Thermo instrument models are also mapped to PSI-MS `NT=...;AC=...` values when recognized.

9 Selected Ion Intensity Visualisation

The **MS2 Selected Ion Intensity** plot keeps the PSI-style column name used in the scan-header exports. The value represents the intensity of the selected precursor ion associated with an MS2 scan, plotted against retention time. It is therefore distinct from MS2 total ion current, which summarizes the fragment-ion signal in the MS2 scan itself.

For robustness across Thermo RAW files, MetaXtract resolves this value in two steps. First, it checks the RAW trailer metadata for **Selected Ion Intensity** or the legacy source alias **Precursor Intensity**. If that value is missing, MetaXtract falls back to **GetPrecursorIntensityFromScanNumber**, which estimates the precursor intensity from the corresponding precursor/master scan and monoisotopic precursor m/z . When fallback values or missing values are used, the Plotly report adds a note below the affected **Selected Ion Intensity** panels summarizing how many scans came from RAW trailer metadata, how many used the computed fallback, and how many had no usable value. This behavior is important for mixed acquisition types, because DIA or non-standard RAW files may not contain a single selected precursor intensity for every MS2 scan.

10 Runtime and Memory Metrics

MetaXtract reports per-file runtime and memory usage in both GUI and CLI modes. Each processed RAW file receives a status and timing summary. In addition to printing metrics to the GUI log or terminal, MetaXtract writes a run-level TSV:

```
runtime_summary_YYYYMMDD_HHMMSS.tsv
```

The table contains one row per RAW file with the following fields: `raw_file`, `sample_name`, `status`, `runtime_s`, `start_memory_gb`, `end_memory_gb`, `peak_memory_gb`, and `memory_change_gb`. Memory is measured as resident set size (RSS) for the MetaXtract process and therefore includes Python, native libraries, and Thermo/.NET allocations used during processing.